Foundry Operations Algebra — Foundations

Tomás Ampuero

November 3, 2026

***Abstract*—Foundry Operations Algebra defines domain-independent concepts and correctness guarantees for how laboratory work changes authoritative state. Entities, relationships, specifications, tasks, events, and effects separate modeled identity, intended work, evidence, and accepted change. Versioned task lifecycles, checkpoints, and domain-governed commitments constrain valid transitions, while composition distinguishes independent operations, supported atomic effect groups, and ordered dependencies. A systematic derivation method guides new domain models without prescribing storage, service, or orchestration mechanisms. Foundational conformance obligations and a generic uncertain-execution example make authority, failure, retry, and recovery semantics testable independently of inventory. Domain specializations may add constraints but must not weaken these guarantees.**

***Index Terms*—Foundry operations algebra, Domain modeling, State transitions, Execution lifecycle, Effect commitment, Conformance, System design**

I. Purpose and Scope

Define a foundational reference for how laboratory work changes system state. The document has three responsibilities:

1) **Conceptual data modeling:** Define primitives, relationships, and identity boundaries from which domain models and future schemas can be derived.
2) **Operational correctness:** Define valid transitions, invariants, commitment rules, and the treatment of failure, uncertainty, retries, and corrections.
3) **System-design constraints:** Establish authority and coordination boundaries while leaving implementation mechanisms open.

This document defines the common modeling language and correctness guarantees for laboratory domains, including instruments, analytical results, procurement, scheduling, and material inventory. It does not define any one of those domain models.

A. Reasoning Structure

The document proceeds from primitives and state, through task lifecycle and composition, to domain derivation and domain-independent conformance. Definitions establish meaning; invariants establish guarantees; worked cases test their consequences. Formal completeness, internal consistency, and derivability take priority over brevity.

B. Architectural Principle

Required semantics, open mechanisms: A conforming implementation preserves the defined identities, invariants, authority boundaries, and failure semantics. This document does not require a particular database, service decomposition, event-storage pattern, orchestration engine, or algebra interpreter. A logical domain denotes responsibility for state and its rules, not necessarily a deployed service.

Scheduling, reservations, procurement and suppliers, instrument automation, and specialized scientific processing remain capabilities with defined interfaces. Customer experiments and service orders may supply work translated into internal tasks and workflows.

C. Specialization Contract

The foundational algebra defines the general semantics and correctness requirements for modeling laboratory operations. Domain models specialize its primitives, state structures, operations, and invariants. A domain specialization may introduce additional constraints but must not weaken or contradict the foundational guarantees.

The companion Foundry Inventory — Material & Inventory Operations is one such specialization. The two documents share v1.7 as their compatibility baseline; a domain revision must identify the foundation version against which it claims conformance. Changes to a domain do not by themselves require changes to the foundations.

II. Fundamental Primitives

We begin with six conceptual primitives.

Primitive	Definition
Entity	An identifiable thing with a lifecycle
Relationship	A typed association between identifiable things
Specification	A versioned definition of structure, constraints, or behavior
Task	An independent unit of intended work
Event	An immutable record of something that occurred
Effect	A proposed or committed change to system state

These are conceptual building blocks, not necessarily six generic database tables. Entity identity remains stable through permitted state changes. Identity reuse, creation, and amendment follow explicit versioned Specifications rather than implicit merging based on matching attributes. Relationship endpoint types, cardinality, and temporal constraints are defined by each specialization.

Each Specification must declare its applicability, version-binding, and reevaluation policy: which entities, executions, and authorizations it governs; when a version is bound; and whether a version change retains existing bindings or requires reassessment. The policy must address in-flight executions and existing entities, decisions, and authorizations,

including reassessment triggers and timing and any permitted rebinding. Reassessment or rebinding must be authorized and recorded with the prior versions and history preserved. There is no universal rule requiring either automatic upgrade or indefinite use of an earlier version; a new version must not silently reinterpret existing records.

- **Entities and relationships** describe the modeled world.
- **Specifications** define contracts and constraints.
- **Tasks** describe intended work.
- **Events** record what happened.
- **Effects** describe changes to system state.

Events may occur independently of tasks. A task may produce zero, one, or multiple effects.

Intended effects are distinguished from effects actually committed to authoritative state.

These primitives describe conceptual roles, not disjoint storage types. A TaskSchema is a Specification; its TaskExecution has Entity identity and records Events and Effect references. A recorded decision may likewise have stable Entity identity while its commitment changes decision state. Domain derivation must make these roles and their connections explicit (Section VII).

III. State and Authority

Let S represent the logical state of the system.

Use distinct notation for permissions and proposals:

- $\mathcal{F}$: The set of permitted effect families; each family defines a kind of change and its payload contract.
- δ: An individual proposed effect, with its logical identity, family, affected state, concrete change, and origin and evidence references required by the domain contract.
- Δ: A proposed batch of concrete effects.

An effect family is not itself a proposed state change. A proposed effect becomes committed only after authoritative validation and acceptance; proposing it does not by itself apply the described change.

An operation proposes an effect batch Δ:

$$\text{apply}(S, \Delta) \to \begin{cases} S' & \text{if valid} \\ \text{Error} & \text{otherwise} \end{cases}$$

Let $\mathcal{I}$ represent the system's invariants.

For every accepted transition:

$$\mathcal{I}(S) \wedge \text{Valid}(S, \Delta) \Rightarrow \mathcal{I}(S')$$

An operation may change system state only when the relevant domain invariants remain satisfied.

The domain responsible for the affected state validates and commits its effects. Validation must apply to the authoritative state against which commitment occurs; concurrent changes must not allow an invalid transition merely because an earlier check succeeded.

This state model follows the architectural principle in Section I.B.

A. Historical and Observed State

We distinguish three concepts:

Concept	Meaning
Recorded history	The immutable record of what the system previously recorded
Authoritative state	The current system representation after valid domain transitions and corrections
Observed physical state	Measurements or observations about the actual laboratory

These may disagree.

A correction changes the authoritative representation without rewriting recorded history.

A physical observation provides evidence about reality but does not automatically change authoritative domain state.

References connect observations, corrections, and affected domain records.

B. Domain State and Authority

For modeled domains indexed by k in K, let S_k denote each domain's authoritative state and let T denote task and workflow definitions, executions, and history. A logical view is:

$$S = \left((S_k)_{k \in K}, T \right)$$

Each state component has a declared authoritative owner and applicable invariants. The notation does not imply one physical store, disjoint records, or a globally atomic snapshot.

An operation identifies its proposed effects, affected state, validating authority, and required evidence. Task and workflow coordination depend on the owning domain's commitment, not on a local assumption of success. Dependencies outside one supported transaction boundary follow Section VI.D.

Instrument, decision, and result state are possible specializations. Their domain-specific structures do not redefine the general algebra.

IV. Task and Workflow Algebra

A. TaskSchema

A TaskSchema is a versioned contract describing an operation.

$$\text{TaskSchema} = (I, O, R, \mathcal{F}, L)$$

Where:

- I: Typed input ports, including domain entities, data, and references.
- O: Typed output ports, including domain entities, data, and decisions.
- R: Resource requirements and preconditions.
- $\mathcal{F}$: Permitted effect families, not individual proposed effects δ or proposed batches Δ.
- L: Mandatory lifecycle and effect-commitment contract.

Every TaskSchema defines L, directly or by referencing a versioned standard lifecycle specification.

$$L = (\text{States}, \text{Transitions}, \text{Checkpoints}, \text{CommitRules})$$

Tasks may require domain entities, people, instruments, or external capabilities.

Typed ports may reference domain Specifications to define acceptable inputs and expected output identities.

TaskSchemas declare resource requirements without implementing scheduling or reservations.

Tasks are not fundamentally classified as physical, virtual, manual, or automated.

B. TaskExecution

A TaskExecution represents one invocation of a TaskSchema. It records:

- TaskSchema identity and version, applicable Specification bindings, and any reassessment or rebinding history.
- Actual input bindings.
- Actors and external execution references.
- Current lifecycle state and transition history.
- Checkpoint evaluations and supporting evidence.
- Execution events.
- Actual outputs.
- Proposed and committed effect references.
- Failure, interruption, cancellation, and recovery history.

A task defines an operation. Its execution records what happened. Its effects describe what changed.

The execution/effect independence principle (Section V.E) governs the interpretation of its lifecycle status and physical consequences.

C. Workflows

A WorkflowSchema composes tasks through dependencies and typed input/output connections.

For example:

$$\text{AcquireData} \rightarrow \text{ReviewResults} \rightarrow \text{PublishResults}$$

A WorkflowExecution represents an actual execution of that graph.

Tasks may execute independently of workflows.

The planned workflow remains distinct from its execution history, allowing retries, deviations, and failures.

Workflow dependencies may require task completion, a committed domain effect, or another explicitly defined condition.

Resource feasibility and scheduling may be delegated to external capabilities.

V. Task Lifecycle and Effect Commitment

Every TaskSchema defines how its executions progress, which conditions must be verified, and when effects may be proposed.

The mandatory lifecycle contract defined by TaskSchema is expanded below.

A. States

States defines the possible execution states.

A standard lifecycle may include:

State	Meaning
Pending	Execution created but not started
Running	Execution underway
AwaitingEvidence	Execution outcome or required conditions cannot yet be established
Completed	Required completion conditions satisfied
Failed	Execution cannot complete as intended
Canceled	Execution stopped under an authorized cancellation process

These are execution states, not authoritative domain outcomes: an external action may physically complete while its TaskExecution remains in AwaitingEvidence.

B. Transitions

Transitions defines allowed changes between execution states and their conditions.

For example:

$$\text{Pending} \rightarrow \text{Running}$$

$$\text{Running} \rightarrow \{\text{Completed}, \text{Failed}, \text{Canceled}, \text{AwaitingEvidence}\}$$

$$\text{AwaitingEvidence} \rightarrow \{\text{Running}, \text{Completed}, \text{Failed}, \text{Canceled}\}$$

These are example transitions. Each TaskSchema defines or references its actual rules.

A cancellation request does not establish that an external instrument stopped.

An instrument's success response does not establish that required domain effects were committed.

C. Checkpoints

Checkpoints defines logical verification boundaries during execution.

A checkpoint specifies what must be established before certain transitions or effect proposals are permitted.

Checkpoints may evaluate:

- Instrument results.
- Scientific observations.
- Required execution outputs.
- Domain and operational preconditions.
- Authorization assessments.
- Confirmation of domain effect commitments.

A checkpoint may be satisfied, unresolved, or failed.

Checkpoints are distinct from lifecycle states and need not occur in one fixed sequence.

An external action may be confirmed through an instrument result or authorized physical verification.

D. CommitRules

CommitRules defines when an execution may propose concrete effects from its TaskSchema's permitted families and which committed outcomes are required for completion.

For an individual proposed effect δ and execution E:

$$\text{MayPropose}(\delta, E, L)$$

is true only when the effect's family belongs to $\mathcal{F}$ and applicable lifecycle and checkpoint conditions hold.

Permission to propose an effect is not permission to commit it.

The authoritative domain independently validates the concrete proposal's payload, required evidence and authorization, and applicable invariants against current state:

$$\text{CommitDomain}(S, \delta) \to \begin{cases} S' & \text{if valid} \\ \text{Error} & \text{otherwise} \end{cases}$$

CommitRules distinguish required effects from optional effects.

Task completion requires satisfaction of its defined completion conditions, not acceptance of every possible optional effect.

E. Execution and Effect Independence

Task lifecycle and domain effect commitment are distinct.

A TaskExecution has its own lifecycle state.

Individual effects may be pending validation, committed, rejected, or unresolved.

In particular:

$$\text{Status}(E) = \text{Failed} \not\Rightarrow \Delta_{\text{committed}}(E) = \varnothing$$

Failure, cancellation, or uncertainty does not automatically reverse committed effects.

Nor does it establish that no physical activity occurred.

Task completion cannot substitute for required domain effect commitments.

F. Interrupted and Uncertain Execution

An external operation may physically occur without definitive system confirmation.

When its outcome is uncertain:

1) Known events and observations are retained.
2) The unresolved execution state is represented explicitly.
3) Unverified physical outcomes are not committed as confirmed facts.
4) Non-occurrence is not assumed from missing acknowledgment.
5) Unsafe repetition of physical commands is prevented.
6) Resolution occurs through further evidence, authorized observation, or reconciliation.

Confirmed partial physical activity must be represented even if the overall task fails.

Unresolved outcomes may require restrictions on dependent operations until an acceptable authoritative state is established.

G. Retries and Idempotency

Software retries and physical retries have different semantics.

Repeated commitment requests for the same logical effect must not duplicate state changes.

Repeating an instrument command may perform an additional physical action.

Idempotent effect commitment does not imply idempotent physical execution.

A physical command may be retried only when its lifecycle contract permits it based on known conditions, or when explicitly authorized as a distinct action.

H. Execution Invariants

Invariant	Requirement
Lifecycle conformance	Execution transitions follow the versioned TaskSchema lifecycle
Checkpoint validation	Checkpoints are satisfied only when required conditions are established
Effect permission	Effects are proposed only when allowed by CommitRules
Domain authority	Owning domains validate and commit authoritative effects
Completion integrity	Required checkpoints and effects are resolved before completion
Failure integrity	Failure does not implicitly reverse committed effects
Retry safety	Uncertain physical actions are not blindly repeated
Traceability	Execution transitions, checkpoint results, and effects remain auditable

VI. Operation Composition and Transaction Boundaries

Operations may be composed to express independence, atomic effect commitment, and ordered dependencies.

The algebra provides three conceptual operators.

Operator	Semantics
$O_1 \oplus O_2$	Independent composition
$O_1 \otimes O_2$	Atomic effect commitment
$O_1 \to O_2$	Ordered dependency

These operators express semantic guarantees, subject to the architectural principle in Section I.B.

A. Independent Composition

$$O_1 \oplus O_2$$

The operations have no declared ordering or atomicity dependency.

Either may succeed or fail independently.

Independence does not guarantee conflict-free concurrent execution when operations affect shared state.

All domain invariants remain applicable.

B. Atomic Composition

$$O_1 \otimes O_2$$

Atomic composition selects explicit state-effect batches Δ_1 and Δ_2 proposed by O_1 and O_2 for joint validation and commitment within one supported transaction boundary. The notation does not make arbitrary operations atomically composable.

$$\text{Commit}(\Delta_1, \Delta_2) = \begin{cases} \text{CommitBoth} & \text{if valid} \\ \text{CommitNeither} & \text{otherwise} \end{cases}$$

CommitBoth and CommitNeither refer only to the selected effect batches, not to complete operation lifecycles or unselected effects.

For example:

$$\text{CreateEntity}(e) \otimes \text{LinkEntity}(e, r)$$

Here the relationship references the Entity created in the same effect group. The owning domain may commit creation and relationship establishment together.

An atomic group is valid only where the participating state changes are covered by a supported transaction boundary.

Atomic effect commitment does not imply atomic physical execution.

C. Ordered Composition

$$O_1 \rightarrow O_2$$

The second operation depends on a defined completion condition of the first.

The condition may be:

- A committed domain effect.
- A satisfied execution checkpoint.
- A completed TaskExecution.
- Another explicitly defined dependency condition.

Ordered composition does not imply atomicity or automatic rollback.

For example:

$$\mathrm{AuthorizeResult}(R) \rightarrow \mathrm{PublishResult}(R)$$

Publication depends on the result's committed authorization.

If authorization is committed but publication fails, authorization remains committed while delivery requires resolution.

The coordinating task or workflow records the incomplete sequence and manages resolution.

D. Composition Across Transaction Boundaries

Operations may declare atomic effect groups and inter-operation dependencies.

Atomic groups are committed within a supported domain transaction boundary.

Dependencies spanning transaction boundaries are coordinated through TaskExecution or WorkflowExecution, with explicit completion, failure, and unresolved outcomes.

Execution/effect independence (Section V.E) and uncertainty rules (Section V.F) apply across these boundaries.

E. Composition Invariants

1) Independent composition does not imply transactional coupling.
2) Atomic effect groups commit entirely or not at all within their supported transaction boundary.
3) Ordered dependencies must be satisfied before dependent operations proceed.
4) Ordered composition does not imply automatic rollback.
5) No composition operator makes physical laboratory actions reversible.
6) Failed or uncertain compositions retain confirmed effects and explicitly represent unresolved consequences.
7) Repeated execution and commitment requests remain subject to idempotency and retry rules.

The operators are notation for these guarantees, not additional primitives.

VII. Deriving Domain Models

A domain model is a specialization of the primitives and state-transition rules, not an independent set of guarantees. Use the following method when introducing a new domain.

1) **Bound the state and authority.** State which facts the domain owns, which it observes or references, and which invariants its accepted transitions must preserve. Identify dependencies on other domains without assuming shared transactions.
2) **Establish identity and relationships.** Identify the domain distinctions, identity boundaries, and relationships that must be represented. Determine where stable identity, independent lifecycle, and historical traceability are required. Evaluate relevant alternative representations using Entities, Relationships, and versioned Specifications, making their assumptions and trade-offs explicit. Select a domain model that preserves the required distinctions and correctness guarantees without assuming that every domain concept requires a corresponding software entity. In the selected model, define reuse-versus-creation rules and Relationship endpoint types, cardinality, temporal scope, and constraints.
3) **Separate evidence from change.** Specify Events and observations, their provenance, and the Effects that can change authoritative state. For every operation, declare inputs, preconditions, effect families, outputs, validating owner, authorization, and evidence. An observation is not automatically an accepted state change.
4) **Bind operations to execution.** Define TaskSchemas with typed ports and mandatory lifecycle contracts. Record applicable version bindings and the declared triggers for reevaluating in-flight executions, existing entities, and prior authorizations. Identify checkpoints, required and optional effects, and explicit dependency conditions. Distinguish physical command identity, execution identity, and logical effect identity so retries cannot silently repeat an action or commitment.
5) **Specify commitment and recovery.** Declare supported atomic effect groups and cross-boundary coordination. Define rejection, partial completion, unresolved outcomes, permitted retries, and authorized correction or reconciliation under the execution principles in Section V.E–Section V.G.
6) **Demonstrate conformance and traceability.** Provide valid and invalid transitions, including concurrency and failure cases. Map each proposed schema element or architectural responsibility to a concept, state owner, invariant, and test. Explain why the selected representation is sufficient to preserve the required semantics and guarantees. Conformance evaluates correctness, not structural similarity; the

conceptual model does not uniquely determine an implementation. Retain those links as the implementation evolves.

The derivation deliverable is a reviewable set of definitions, operation contracts, invariant checks, and conformance cases. Storage tables, services, and orchestration mechanisms are subsequent design choices under Section I.B.

A. Example: Result Authorization

A result domain can define a Result Entity with immutable content versions, review Relationships, and a versioned release Specification. An independently received review is an Event; it is not itself release authority. A ReviewResult TaskSchema requires evidence and authorized judgment before proposing a release Effect for an exact result version.

The owning domain validates and commits that Effect. An invariant forbids release without the required evidence and authority; another prevents approval of one version from silently releasing another. Publication in a separate domain depends on the committed release, not merely on task success. If publication fails, the release remains committed and delivery remains unresolved; retries must not create duplicate publications. This derivation does not depend on a quantity-accounting model.

VIII. General Invariants and Conformance

The execution invariants (Section V.H), composition invariants (Section VI.E), state-validation rules (Section III.B), and specialization contract (Section I.C) apply to every domain. The obligations below test those guarantees independently of any material or inventory model.

A. Foundational Conformance Obligations

Obligation	Required test and outcome
F-01 — Identity and versions	Exercise identity reuse, creation, and amendment under pinned Specifications. Permitted changes preserve Entity identity and prior assertions; changed rules do not silently reinterpret existing references.
F-02 — Authority and valid state	Attempt an unauthorized effect and a stale-precondition commitment after a concurrent change. The owning domain prevents an invalid authoritative transition.
F-03 — Evidence and history	Record an observation and correct an erroneous assertion. Evidence alone does not authorize a state change; correction preserves the original history and its links.
F-04 — Lifecycle and checkpoints	Attempt an undeclared transition and a transition whose checkpoint is unresolved or failed. Neither is accepted as satisfying the required contract.
F-05 — Proposal and completion	Propose a disallowed effect; then reject or leave unresolved a required effect. Proposal permission and domain acceptance remain distinct, and completion is blocked until required outcomes are satisfied. Optional effects follow the contract's explicit completion rules.
F-06 — Failure and cancellation	Fail or cancel an execution after a valid commitment. The commitment remains authoritative; cancellation requests do not prove that external activity stopped.
F-07 — Uncertain execution	Withhold acknowledgment or completion evidence. Non-occurrence is not inferred, unverified consequences remain unresolved, and unsafe physical retries are prevented.
F-08 — Retry identity	Repeat the same logical effect request. No state change is duplicated. A repeat physical command is evaluated separately under its lifecycle contract rather than assumed safe.
F-09 — Composition	Exercise independent operations, an invalid atomic group, and an unmet ordered dependency. Independence does not bypass shared-state validation; atomic groups commit all or none; a dependent operation cannot proceed early.
F-10 — Transaction boundaries	Fail a later operation across an authority boundary. Earlier commitments remain valid, unresolved work is represented, and unsupported cross-boundary atomicity is not claimed.
F-11 — Traceability	Reconstruct an execution's Specification version, inputs, actors, transitions, checkpoint evidence, outputs, and effects without relying on its final status alone.
F-12 — Specialization	Review a proposed domain rule that relaxes a foundational guarantee. Reject the claimed conformance; domain constraints may strengthen but not weaken these obligations.

B. Generic Example: Uncertain External Execution

An instrument receives a command for one irreversible operating cycle. Communication times out before its physical outcome is established. The TaskSchema requires separate checkpoints for command acceptance, verified outcome, owning-domain commitment, and execution resolution.

The execution enters AwaitingEvidence. Command acceptance is retained as evidence, not proof of physical completion. No unverified success Effect is committed, no absence of acknowledgment is treated as proof of non-occurrence, and the physical command is not blindly repeated.

1) **Confirmed full outcome:** The owning domain validates and commits the evidenced Effect once. Completion still requires the contract's remaining checkpoints.
2) **Confirmed non-occurrence:** No physical-outcome Effect is committed; retry or failure follows the lifecycle's declared rules.
3) **Confirmed partial outcome:** Established consequences are recorded using the domain's permitted effect families. The intended cycle may fail without erasing those consequences.
4) **Unknown outcome:** Evidence is retained, dependent operations remain restricted as required, and resolution requires further evidence or authorized reconciliation rather than an invented outcome.

A duplicate confirmation must not duplicate a commitment. A later failure must not reverse established consequences implicitly. This case tests F-05 through F-10 without assuming a particular domain's entities or accounting rules.

C. Validation Method

A complete executable case declares its initial state, versioned TaskSchema and lifecycle, dependency conditions,

events and evidence, proposed and committed effects, expected state, applicable invariants, and invalid variations. Include concurrency, partial failure, uncertainty, and recovery alongside the success path.

These are conformance obligations and a worked semantic example, not a claim that executable implementations or tests already exist. Every implementation must supply evidence for the applicable obligations. Every domain specialization adds its own cases; passing those cases is not a substitute for foundational conformance.

IX. Conclusion

A conforming design provides three reviewable artifacts:

1) **A derivation map:** Domain concepts and schema elements traced to primitives, Specifications, identity boundaries, and state ownership.
2) **A correctness contract:** Operations, lifecycle checkpoints, effect commitments, invariants, and recovery rules stated independently of implementation mechanisms.
3) **Conformance evidence:** Tests of accepted and rejected transitions, authority, evidence, uncertainty, retries, corrections, and composition.

The framework requires valid, auditable, domain-authorized transitions. It does not impose a universal quantity-conservation or balanced-posting rule; such constraints belong to the domain specializations that need them. Database schemas, service boundaries, scheduling strategies, and orchestration mechanisms remain open under Section I.B.

The foundations are extended only when an important scenario cannot be represented without changing their semantics. Refining a particular domain's entities, policies, or accounting rules is normally a revision of that specialization.

The objective is to model authoritative change while preserving historical integrity and operational correctness.